

Driving Value and Efficiency Through Test Automation: A Strategic Proposal for company

Executive Summary	3
1. Current State of Quality Assurance at company	3
1.1. Key Challenges & Their Impact	3
2. The Vision: Embracing Test Automation for company	5
2.1. Our Vision for Automation at company	5
How We'll Make This Happen	5
Key Pillars of Our Automation Vision	6
3. Gap Analysis: Where We Are vs. Where We Want to Be	7
3.1. Skills and Expertise	7
3.2. Tools and Infrastructure	7
3.3. Processes and Methodology	8
3.4. Data Management	8
4. Recommended Solution: Our New Automation Project	9
4.1. company Turbo Automation Objective	9
4.2. How company Turbo Automation Drives Strategic Goals	10
4.3. Strategic Pillars of company Turbo Automation	10
4.4. The In-Scope of company Turbo Automation	13
4.5. company Turbo Automation - Solution Framework	14
4.5.1. Internal Hierarchy of the Test Automation Framework	14
4.5.2. External Interaction: The Automation Framework within the CI/CD Pipeline	15
4.6. Company Turbo Automation - Team Structure	17
4.6.1. Collaboration with Manual Testers Throughout the Project Lifecycle	17
5. Alternative solution and evaluation criteria	19
6. Final Word	19
Appendix A : Project Charter: company Turbo Automation	20
Project Title:	20
Project Purpose/Justification:	20
Project Objectives	20
Project Scope Statement	20
Key Deliverables	21
Key Stakeholders	21
Appendix B : Proof-of-Concept, Hypothesis validation	22
B-1 SUT "My Learning Platform" project :	22
B-2. Selenium-Python UI Quality Project: The Core of Our Proposal	23
B-3. CI/CD Pipeline Integration: A Unified Value Stream	23
Appendix C : Connecting Concepts	25

Executive Summary

This document addresses a key plan to incorporate Automation Testing Practices into our value stream pipeline for a company that plans to shift into automation. We believe automation is a game changer, and implementing it with CI/CD requires a cultural shift in the software development life cycle we follow in company. Automation will speed up our development, make our products better, and help us work smarter with on-push feedback. We point out our current testing approach, share a clear picture of what an automated future could look like, and highlight the significant gap between our current situation and our automation vision.

We then present our proposed solution to bridge this gap, enhancing our engineering process capability and establishing a clear value-delivery pipeline. This plan includes a Proof of Concept (PoC) to validate the benefits of automation for a company, and at the same time, the constraints and the requirements needed to succeed in Automation, detailing the necessary resources (tools, human capital, and required skills). Following this, we will explore alternative solutions and their respective degrees of automation.

Finally, the document provides appendices outlining how to form and develop the automation team, manage potential risks, and detail key role activities, including the essential 'test-first' practices the automation team should adopt in collaboration with the manual testing team.

1. Current State of Quality Assurance at company

The company's QA team demonstrates remarkable dedication and diligence across our diverse project portfolio. However, our reliance on manual testing and traditional QA processes introduces substantial challenges that actively impede our operational efficiency and the velocity of our software delivery.

1.1. Key Challenges & Their Impact

- Late Feedback Cycles:

Defects are often discovered much later in the development process, the QA team needs significant time to do the test for the new builds across different projects. This leads to slow developer iteration, delayed decision-making for Project Managers due to lack of project health status, and ultimately, reduced customer satisfaction due to prolonged issue resolution.

- Ad-Hoc Regression & Hidden Bugs:

Our regression testing (re-verifying existing features) is left in most cases and in some cases is performed manually and inconsistently (not formally, we personally focus to hit only the positive path of the core business flows). The absence of formal traceability increase the probability of hidden bugs to be discovered in production environment

- Incomplete Test Coverage:

Critical testing areas such as Unit, Integration, API (crucial for our Django services), and Performance testing are largely missing or underdeveloped. This leaves essential software layers and functionalities inadequately checked, leading to potential vulnerabilities and instability.

- Informal Early-Stage QA:

Our early-stage testing efforts (e.g., "Shift-Left" activities and static analysis) are largely informal and unsystematic, it is just a form of review of the test bases, especially SRS. This pushes defect discovery and their associated remediation costs much later into the development cycle, increasing overall project expenses.

- High Resource Allocation for Repetitive Tasks:

Our highly skilled QA team spends an excessive amount of time on mundane, repetitive manual tasks. This significantly limits their capacity to perform more strategic, high-value testing activities such as complex exploratory testing or in-depth scenario design.

- Shifting from a Third-Party Perception to Shared Quality Ownership:

Currently, the QA team is often treated like an independent, third-party group. While we acknowledge the importance of an independent QA function, we firmly believe the responsibility for

quality needs to be propagated and owned throughout the company. This isn't about assigning blame but fostering a company-wide commitment to delivering high-quality products at every stage of the development lifecycle.

2. The Vision: Embracing Test Automation for company

Test automation uses specialized software tools to run predefined tests automatically, without people doing it by hand. Its main goal is to quickly and accurately check software features and performance, repeatedly per builds. This cuts down on manual work, speeds up the whole testing process, and makes testing more consistent.

2.1. Our Vision for Automation at company

For companies, test automation is the key to completely changing how we deliver software. We plan a slow, steady, but definite move from our current manual testing to a smart, forward-thinking, and very efficient quality engineering team. This change will help us deliver better software faster and with more confidence, directly adding more value across all our business operations.

We aim to make "Reducing Lead Time" a main goal for our daily work. This will directly help us reach our bigger strategic goal of "Faster Time to Market."

How We'll Make This Happen

To achieve this vision, we need a well-defined and documented process in the QA department that clearly spells out:

1. Which Projects to Automate: The steps and rules for choosing new projects that are good candidates for automation.
2. Teamwork and Roles: How all the different roles in a project's development pipeline will work together, and what each person's responsibilities are.
3. Combining QA Efforts: How automated testing, and manual testing will work together for each project.

Our success also depends on having skilled and dedicated QA automation specialists who will:

- 1. Build Testing Tools: Create strong testing tools for projects, which will be a long-term strategy for the company.
- 2. Work with Others: Collaborate effectively with developers, DevOps, Project Managers, business teams, and manual QA to help software get delivered smoothly.
- 3. Keep Improving: Use the "Plan-Do-Check-Act" (PDCA) method to constantly improve and update our automation tools, including using new technologies like AI.

Beyond this, our vision includes:

- 1. Smooth Delivery: Making sure all main and repeated project tasks are delivered consistently and predictably.
- 2. Focus on Risks: Using risk-based testing to decide which important tasks should be automated first.
- 3. Integrate Automation: Putting our automated test code directly into the different stages of our software delivery process.
- 4. Quick Reports: Quickly sharing updates on the health of our software builds with everyone who needs to know.

Key Pillars of Our Automation Vision

We can summarize the key pillar we are going to build as the table below

Pillars	How	Impact
Accelerated Release Cycles	automating repetitive and time-consuming tests	deploy updates and new features with greater speed and confidence
Enhanced Software Quality	replaces ad-hoc regression with systematic, comprehensive	Reducing the risk of hidden bugs
Optimized Resource Utilization:	Automating mundane, repetitive tasks frees up our highly skilled QA team.	focus on more strategic, high-value activities such as complex exploratory testing, usability analysis

Scalability & Maintainability	We can expand test coverage and execution capacity without a linear increase in manual effort	Adapt quickly by adding on a stable foundation
-------------------------------	---	--

3. Gap Analysis: Where We Are vs. Where We Want to Be

This section highlights the big differences between our current manual testing and our goal of using automated quality checks. While other teams are involved, we're focusing on the key changes our QA team needs to make to collaborate and make automation happen. Our analysis shows four main areas where we need to bridge gaps:

3.1. Skills and Expertise

Where We Are Now: Our QA team is strong in manual testing and knows our applications well, including Django systems. But we have limited hands-on experience with automation tools, coding for automation (like Python), and integrating continuous testing into our CI/CD pipelines.

Where We Want to Be: We want a QA team that can design, build, run, and maintain automated tests across all software layers—from user interface to code units (UI interaction unit). This means understanding and smoothly integrating the right automation tools into our current development process.

The Gap: To close this, we need to invest significantly in training our current QA team to become automation specialists. We might also consider hiring new talent with proven automation expertise to speed things up and provide initial guidance.

3.2. Tools and Infrastructure

Where We Are Now: Our current testing setup is mainly for manual work. We clearly lack proper tools for test automation, smart systems to manage our tests, good ways to handle test data, and scalable environments needed for running automated tests. Because of this, our development process (CI/CD) barely has any automated testing, if at all.

Where We Want to Be: We want to set up a full set of automation tools, including:

- UI Automation: Strong tools to regularly test our web and mobile apps (like using Selenium for our websites).
- API Testing: Special tools to check how our services talk to each other and how our backend logic works (these will fit well with our Django APIs).
- Performance Testing: Solutions to check if our systems can handle different amounts of work and respond quickly.
- Test Management System (TMS): A central place to organize all our tests and track results.
- Version Control: A solid system (like Git) for managing and working together on automation scripts.
- CI/CD Integration: Directly and smoothly putting automated tests into our continuous development and deployment process.
- Dedicated Test Environments: Stable, separate, and consistent environments made specifically for running automated tests.

The Gap: To close this, we need to strategically invest in and carefully set up the right automation tools and infrastructure. This means buying the necessary software, carefully setting up dedicated and repeatable test environments, and fully integrating automation into our existing development pipelines.

3.3. Processes and Methodology

Where We Are Now: Our QA process mainly reacts to problems; we usually find bugs late in the development stage. We execute test cases by hand, and they're often tied to specific applications, making them hard to reuse. We only partially do regression testing when we feel we need to, which takes a lot of time and effort.

Where We Want to Be: We aim to move to a "Shift-Left" approach, where we start testing much earlier in the development process. This means:

- "Automation-First" Thinking: Always thinking about how to automate new features and important checks right from the start.
- Standard Test Frameworks: Using consistent ways and tools to build, manage, and run our automated tests.
- Continuous Testing: Running automated tests often and consistently throughout the entire development process.

- Clear Reporting: Getting useful, real-time information from our test results so we can make quick decisions.

The Gap: We need a fundamental change in how we do QA. This means standardizing our automation practices, bringing automation into the development process earlier, and clearly moving towards continuous testing. This shift requires new ways of working, much better teamwork with development teams

3.4. Data Management

Where We Are Now: We often create test data by hand whenever we need it, and we do it unplanned. This unplanned way of working often leads to inconsistent test results, makes it hard to reliably recreate reported problems, and sometimes uses sensitive information from live systems, which brings up security and privacy risks.

Where We Want to Be: We need a dependable, organized, and automated way to manage our test data. This means being able to automatically create varied, realistic, and reusable sets of data for our automated tests, while at the same time protecting sensitive information by making it anonymous or creating fake data.

The Gap: To close this, we need to put strong plans in place and get the right tools for managing our test data effectively. This is extremely important to make sure our automated tests are trustworthy, always repeatable, and cover all necessary situations without risking privacy or security.

4. Recommended Solution: Our New Automation Project

Achieving our automation vision requires a clear shift from our current approach. The significant variance between the company's current operational attitude towards automation and our stated strategic vision represents a critical gap that demands immediate, structured action.

To close this gap and quickly advance our automation, we must formalize our effort. We will launch Turbo Automation, a dedicated development project led by the Quality Department. Its main goal is to drive the widespread use of strong automation testing across all parts of the

company. This project will make us more efficient, improve product quality, and fundamentally change how we operate.

4.1. company Turbo Automation Objective

- 1. Core Automation Team Establishment & Initial Capability:
- 2. Foundational Automation System & Tools Setup:
- 3. Initial Automation of Software Delivery Components
- 4. Establish and maintain comprehensive automation documentation

The specific outputs of company Turbo Automation may evolve based on future decisions, but initially, the project aims to deliver:

- 2. A Mature, Ready-to-Use Automation Framework: This includes fully integrated tools, clearly defined roles, and robust operational policies (comprising detailed procedures and work instructions as needed).
- 3. A Flexible and Stable Framework for Project Generation: This framework will be designed to easily set up and support new QA automation projects for any candidate identified by the PMO. This QA project will be invested later in the pipeline defined for that project

4.2. How company Turbo Automation Drives Strategic Goals

Our project, company Turbo Automation, is more than a technical upgrade. It is key to reach the company's main company goals. By meeting the objectives in Section 4. and focusing on four key areas (explained below), this project will help company achieve the following important strategic outcomes:

company's Big Goal	How Our Automation Project Helps
Become More Competitive	We can deliver software much faster and react quickly to market changes. This means new features and services get to customers before competitors, giving us a clear advantage.
Strengthen Our Team	We're training our staff and building automation skills inside the company. This means we'll rely less on outside help and become more efficient

	as a team.
Lead Digital Transformation	By automating manual tasks and finding bugs early, we make our operations faster and cheaper. This helps us offer top-notch digital experiences and stay a leader in the digital world.
Improve Customer Experience	Delivering higher quality software faster, with fewer problems, directly means happier customers. This builds trust and keeps them loyal.

This strategic alignment underscores that our automation initiative is not merely a technical upgrade, but a fundamental investment in the company's future growth, efficiency, and market leadership.

4.3. Strategic Pillars of company Turbo Automation

Our plan is built on four key pillars that will serve as the core objectives for this new internal project:

1. Building Internal Skills:

Skill Area	Description / Objective
Python Programming	Essential for script development in our Django environment.
Pytest Framework	Mastery for designing scalable, maintainable, and efficient test suites.
Selenium WebDriver	Practical expertise in automating complex UI interactions for web applications.
API Testing	Understanding how to test our backend Django APIs and other services using. Postman Test Scripts javascript
Jmeter	Writing Performance test script the check the performance
Version Control (Git)	Best practices for collaborative and systematic management of automation code.
CI/CD Integration	Holistic understanding of how automated tests fit into our pipelines.

2. Smart Tool Adoption (Please refer to the section for alternatives:)

Tool/Technology	Purpose & Benefit
-----------------	-------------------

Python Programming	Our primary scripting language, offering excellent compatibility and a rich ecosystem for Django.
Pytest Framework	Our robust and extensible test framework, providing powerful assertion, test discovery, and reporting.
Selenium WebDriver	For highly effective and cross-browser UI automation of our web applications.
Python Libraries (e.g., Requests)	Integrated with Pytest for comprehensive API testing of our Django APIs.
Apache JMeter	Tools to be considered for performance testing and assessing system scalability.
CI/CD Integration	Holistic understanding of how automated tests fit into our pipelines.

3. Streamlining QA Automation Processes:

We will comprehensively update our QA processes to proactively integrate automation much earlier in the development cycle ("Shift-Left"). This means:

Team

Stage	Shift-Left action	Why It Matters (Benefit)
Kick-off and Planning	QA Team Leader: - Identifies core business functions, critical user paths, high-risk areas, and repetitive flows that are prime candidates for automation. - Collaborates with product and development to define clear, testable requirements from the outset.	Ensures automation efforts are focused and deliver maximum value, proactively identifying what needs automation early on.
Frontend Development	QA Developer: - Works directly with frontend developers to design the Page Object Model (POM) for UI elements. - Collaborates on creating automated tests for UI components as they're built.	Builds testability into the UI from day one, leading to fewer UI-related bugs and faster, more reliable frontend changes.
Backend Development	QA Developer: Partners with the backend team to design and review API endpoints and data structures.	Catches logic and data issues at the API level, preventing larger problems down the line and ensuring data

	• Prepares detailed test cases and synthetic test data tailored for backend automation. • Reviews business logic and expected results to align with API test automation.	integrity.
Integration	QA Team: • Designs and implements automated integration test flows across different services and components. • Ensures data availability and validity for robust end-to-end testing scenarios. • Sets up automated data seeding and cleanup processes.	Quickly finds complex bugs where different parts of the system interact, ensuring the entire application works seamlessly together.

4. Establishing Best Practices (Document the New Process):

What It Is	How It Helps Us	Why It's Important (Benefit)
Consistent Guidelines	Creates clear rules for how all teams build and use automation.	Everyone works the same way, avoiding wasted effort and ensuring smooth connections.
Easy Knowledge Sharing	Acts as our central hub for automation expertise and success stories.	Good ideas spread fast, allowing teams to learn and grow from each other.
Continuous Improvement	Regularly reviews how our automation is doing and finds ways to make it better.	Our automation keeps getting stronger, helping us stay ahead and always improve.

4.4. The In-Scope of company Turbo Automation

Test Level/Tool	Focus	ROL
Unit Testing	Django models and views	Developer
Integration Testing	Django API endpoints and their interactions	Mostly Developer, we can apply pair review (tester touch the structure and learn white box technique)

API Postman	Direct API endpoint testing, request/response validation, contract testing, API functional checks.	Tester (Write Automated API Script) based on clear API specification DRF
JMeter	Performance, load, and stress testing of systems/APIs; can also be used for functional API testing.	Tester (write Jmeter test script)
System Testing	The complete and integrated system against specified requirements.	Tester (write python pytest test cases and data scripting)
CI/CD practices	Inject the QA project (the one developed as System Testing) in Jenkins pipeline	QA Automation Lead: • integrate the Jmeter scripts into pipeline against the Staging environment • Integrate the Postman script into staging environment • Integrate Selenium UI test cases into staging environment • Maintain the framework Architecture and provide the reporting to interested parties (developer, QA, PM...)
Acceptance Testing	Validates the system against business requirements, often with end-users or stakeholders.	A kind of inspection Session: Manual Tester with PM

- *For Unit and Integration Tests: The Development Department will own the decisions for Unit and Integration Tests. While these are primarily developer practices before any code commit, their insertion into the pipeline is crucial for protecting our staging environment.*
- *CI/CD Practices & JMeter Execution: All CI/CD practices must adhere to our environments' policy. Critically, JMeter performance tests should always be run against an environment that closely mirrors production.*
- *Automation Scope Decision: Executive management will ultimately decide which testing levels to apply automation on. However, we present here our ideal automation pipeline for consideration.*

4.5. company Turbo Automation - Solution Framework

To achieve our automation goals, we must understand two key areas of our solution:

1. The architecture of our test automation framework itself
2. How it integrates with CI/CD-value delivered by the project

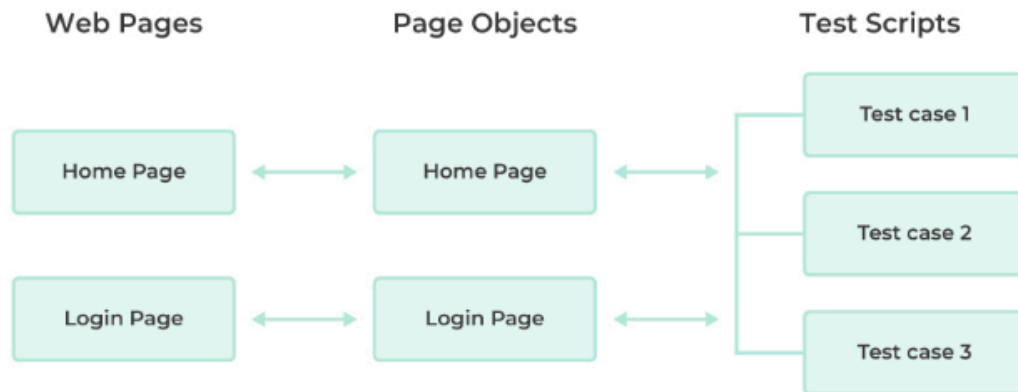
4.5.1. Internal Hierarchy of the Test Automation Framework

To achieve our automation goals, we must understand two key areas: the architecture of our test automation framework, and how it integrates with our CI/CD pipeline. While these aspects are technical, it is crucial to clearly present to management the significant amount of work and specialized skills required to build and maintain the automation-project framework.

The figure below clearly demonstrates the Page Object Model (POM) as a flexible and essential design pattern for our QA project. This pattern primarily serves to enhance test maintainability and reusability by establishing clear separation of concerns:

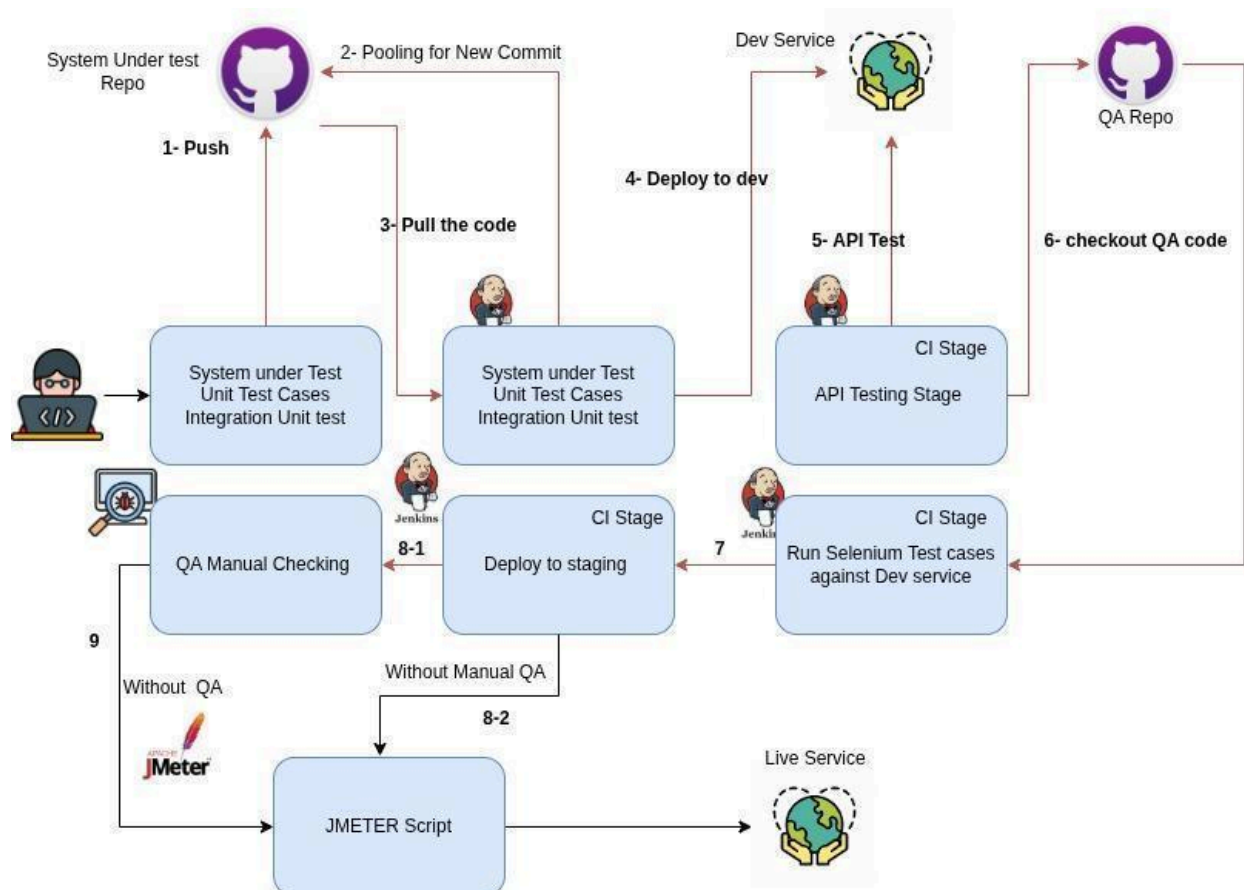
1. **Dedicated Page Object Classes:** Define a specific class (Page Object) for each unique HTML page or major UI component. This class encapsulates all UI elements and interactions unique to that page or component.
2. **Test Classes Utilizing Page Objects:** Create dedicated test classes that implement test cases by interacting with the UI solely through the methods exposed by the Page Objects. This separation ensures that test logic remains clean and isolated from UI implementation details.

An Overview Of The Page Object Model



This framework involves much underlying logic that will be detailed in later documentation.

4.5.2. External Interaction: The Automation Framework within the CI/CD Pipeline



CI/CD Pipeline Stage	Description
1- Developer push commit	• Developers push their code changes to a version control system (e.g., Git). • Remote repo should be configured to hook Jenkins server on push
2- Pooling for new commit	The CI server detects the new commit and automatically triggers a new build. • Either by web hook set on the remote repo • Or by pooling-SCM
3- pull the code	Jenkins job's stage checkout the dev branch and • Build • Run unit tests • Run integration test
4-Deploy to dev service	If all test cases (Unit, Integration) passed, then Jenkins job's stage deploy the dev branch into Dev server
5-API Test	Jenkins job's stage will run Postman API test cases against the dev service
6-checkout QA code	If API Postman all passed, Jenkins job's stage will: 1. Checkout the QA Selenium code 2. Run all Selenium test cases
7-Deploy to staging	If all Selenium test cases passed, then. Jenkins job's stage deploy the dev code of SUT to Staging server Generate full test report Communicate the result as a kind of quick feedback to interested parties
8-1 With Manual QA Inspection	Based on our design decision, we either deploy to staging for QA inspection: • Exploratory Testing, • a kind Of User Acceptance Testing (UAT) Support, • Usability and User Experience (UX) Testing) • Double check for any core business flow as needed
8-2 without Manual QA Inspection	For a some degree of trust and according to the amount of familiarity with software, we could run performance on the staging
9 manual stage	After manual team inspection, a manual action needed here to • Run J meter test script on the staging environment • Merge dev to main and update the live service

When it comes to JMeter testing, while running tests directly on the live service would indeed provide the most accurate real-world performance metrics, this is considered a high-risk practice. Such an approach could severely impact the experience of our actual business users, potentially leading to service degradation or outages. The critical solution, therefore, is to prepare each project's staging environment to be as close a replica of the production environment as technically feasible.

4.6. Company Turbo Automation - Team Structure

We must strongly emphasize the critical need for a dedicated team for automation. As we've highlighted, building a flexible automation framework requires a core team focused solely on this task, at least during this initial phase.

Expecting our QA Automation Engineers to jump between manual testing, daily support, and automation development would truly slow us down. While it's great to be able to handle many tasks, context-switching, as studies confirm, will directly hurt our ability to hit our automation goals. This leads to delays, inefficiencies, and a slower path to realizing our vision. A dedicated focus, on the other hand, keeps us moving forward steadily and allows for the deep expertise needed for successful automation.

Here's how the team will start:

- **Automation Framework Maintainer:** This person will handle the automation system, making sure it runs smoothly and managing how it connects with other tools, as explained in Section 4.5.2. They'll report to the QA team lead and work hand-in-hand with them, especially early in a project. Also, maintainer will review the code to eliminate the technical debt for effective repetitive regression
- **Two QA Selenium-Python Developers:** These two will do the main work: designing and running our automated tests using Selenium and Python.

Initially, these roles will work under the QA Team Lead's oversight. We know there's a chance we'll need a dedicated QA Lead for the Automation team later. That will be a management decision, depending on things like how many automation projects we have compared to all projects, and how mature our initial automation system becomes.

Forming this automation team isn't just about bringing in a few new people; it's about strategically weaving a powerful new capability into our software delivery process. Treating Automation as a One-Time Task, Not an Ongoing Product, will adversely affect the maturity and mastering of the Automation technology

4.6.1. Collaboration with Manual Testers Throughout the Project Lifecycle

Automation enhances manual testing; it doesn't eliminate it. Manual testers transform into strategic partners, focusing their human intuition and critical thinking on complex areas. Automation, in turn, efficiently handles repetitive and scalable tasks. The following table details this collaborative interaction across project phases, highlighting our 'Shift-Left' approach from project inception.

Project Phase	Automation Team Focus/Activities	Manual Tester Collaboration/Contribution
Discovery/Planning/Requirements Gathering (Shift-Left Begins)	Define the overall automation strategy (scope, tools, framework design). Conduct feasibility analysis for new features. Plan test environments and CI/CD integration requirements.	Identify and prioritize test cases suitable for automation. Refine acceptance criteria to be precise. Share crucial business context and domain knowledge with the automation team.
Design/Development (Automation Development Accelerates)	Develop or enhance the automation framework. Prioritize API and Unit test script development (Shift-Left). Manage test data. Integrate automated tests into CI early.	Create detailed manual test cases (including edge cases). Participate in technical discussions and walkthroughs. Provide rapid feedback on early builds. Review API specifications for clarity and testability.
Testing/Iteration (Continuous Testing)	Continuously expand automated regression suites (UI & API). Implement performance testing (JMeter) as core functionality stabilizes. Maintain, refactor, and optimize existing	Lead exploratory, ad-hoc, and usability/UX testing. Handle complex scenarios. Verify defect fixes. Assist automation engineers in test case grooming and debugging automation failures.

	automation. Analyze test results and report defects. Enhance the CI/CD pipeline.	
Release/Deployment	Execute a final, comprehensive suite of automated regression tests on the release candidate. Perform automated smoke/sanity testing post-deployment. Set up automated post-deployment monitoring.	Conduct quick manual smoke/sanity checks in the production environment. Support business users during User Acceptance Testing (UAT).
Maintenance/Post-Release	Continuously monitor automated test trends, flakiness, and coverage. Evolve the automation framework to support new technologies. Optimize the test suite for efficiency.	Assist in reproducing production incidents. Begin exploring new features for future sprints, providing early insights for the next planning cycle.

5. Alternative solution and evaluation criteria

As we mentioned above, implementing our proposed automation solution will require a significant and sustained effort.

This section explores various alternative solutions to achieve our automation goals. To ensure we choose the best path for the company, this will help to conduct "Alternative-evaluation" practice, this will definitely help management to take a clear idea about the effect of each choice against our current status

6. Final Word

Appendix A : Project Charter: company Turbo Automation

This document formally authorizes the "company Turbo Automation" project and outlines its core purpose, objectives, and high-level scope.

Project Title:

company Turbo Automation Project

Project Purpose/Justification:

This project is initiated to fundamentally transform the company's software quality assurance processes. It is an internal project in the QA department. It aims to achieve strategic objectives by accelerating testing cycles, improving software quality, reducing manual effort on repetitive tasks, and enhancing feedback loops across the development lifecycle by using automation techniques to form a stable, flexible and scalable framework by the end of 2025.

Project Objectives

The primary objectives of the company Turbo Automation Project are to:

- Significantly reduce software delivery lead time.
- Improve the speed and effectiveness of defect detection.
- Increase the overall quality and reliability of the company's software products.
- Optimize resource utilization by shifting manual efforts from repetitive to exploratory testing.
- Shorten the feedback loop for faster development and continuous improvement.

Project Scope Statement

- Designing and implementing a comprehensive test automation framework
- Integrating the automation framework seamlessly into the Continuous Integration/Continuous Delivery (CI/CD) pipeline.
- Establishing clear roles and responsibilities for automation within the SDLC.

- Developing initial automated test suites for selected critical functionalities (API, UI regression).

Out of Scope:

- Full automation of *all* existing manual tests in every project immediately.
- Specific test case prioritization will be determined by executive management.
- Security testing, but need to be discussed to know where it fit in the pipeline
- Automation Mobile testing

Key Deliverables

The project is expected to deliver:

- A mature, ready-to-use test automation framework with defined tools, roles, and policies.
- A flexible framework capable of generating and supporting QA automation projects for any candidate identified by the PMO.
- Initial automated test suites integrated into the CI/CD pipeline for select critical areas.
- Documentation for the automation framework and CI/CD integration and other quality documents like work instruction, procedure and checklists as needed..

Key Stakeholders

- Executive Management
- Development Department Leads & Teams
- QA Department Leads & Teams
- PMO (Project Management Office)
- Product Owners / Business Analysts
- IT Operations / DevOps

High-Level Resource & Effort Statement:

Implementing this automation solution will require a significant and sustained effort. This includes not only the initial setup and development but also continuous maintenance and the need for specialized skills over time.

Appendix B : Proof-of-Concept, Hypothesis validation

This appendix details our Proof of Concept (PoC), developed to validate the proposed automation solution. For this PoC, we built a simple Django application simulating an educational platform, which serves as our System Under Test (SUT). Through this SUT, we demonstrate the practical benefits of automated testing using the tools outlined in our solution.

B-1 SUT "My Learning Platform" project :

Our System Under Test (SUT) includes three main user roles: Admin, Teacher, and Student. The core business model generates revenue through a commission applied to courses taught by teachers and enrolled in by students. This commission value is central to the system's financial operations. Based on this, we have these flow

- Student Registration and Profile Management: How students sign up and manage their personal information.
- Teacher Onboarding and Course Creation: How teachers join the platform and create new courses.
- Admin Approvals: How administrators approve new teachers and new courses.
- Teacher Dashboard and Reporting: Functionality for teachers to view their performance, reports, and manage their courses.
- Other General Flows: This includes common interactions like "Contact Us" and core functionalities within the "Student Dashboard."

Comprehensive Test Levels within the SUT:

Our System Under Test (SUT) uses several levels of automated testing to ensure quality:

- Unit Tests: we developed about 10 unit test cases about the core functionality using python-pytest
- Integration Tests: we developed about 5 integration test cases to check the communication between unit code, using python-pytest
- API Tests: We used DRF to generate API end point, which we then import into Postman and write some post script which contain about 13 test cases

- UI (User Interaction) Tests: we use Selenium web driver with python-pytest to write about 9 flows among Students, Teachers and Admin

SUT Operational Environments for Realistic Testing:

To provide a demonstration very close to a real-world situation, we deployed the System Under Test across distinct operational environments, we used the tools below:

- We hosted our SUT on GitHub [here](#) which contain basically two branches, Dev branch: is maintained as a long-term integration point, mirroring a real-world scenario and main branch, which we use to feed the real user environment
- We hosted our SUT service using [Render](#) , which offer a trial hosting for limited time with limited database , we created two services , one for dev and the second for the live

Github	Branch	Render Web services
majd.kassem.business	dev	Dev service
	main	Live service

B-2. Selenium-Python UI Quality Project: The Core of Our Proposal

This project forms the central pillar of our Proof of Concept, specifically designed to visibly demonstrate the immediate value and power of end-to-end automation. It highlights not only *how* automation can test, but also the return on investment (ROI) for the effort required.

- Technology Stack: This is a Selenium-Python UI test project, we used pytest for testing
- Hosted on [Github](#) , we use mainly dev branch just for demonstration
- Implement POM design pattern for automation framework with
- Contain 9 test cases for the core business of the SUT
- Contains Jenkins file which manage the flow of delivering the new commit push

B-3. CI/CD Pipeline Integration: A Unified Value Stream

Our CI/CD pipeline automates the journey from code commit to deployment, creating a unified and integrated process for development and quality assurance. This pipeline is structured as a single Jenkins job with multiple stages. The Jenkinsfile (pipeline definition) is located within the SUT's code repository. We are currently using Jenkins on a local machine to orchestrate this job across its various stages.

Pipeline Stage	Description
Pipeline pooling Github	Every 2 minutes to check any new commit pushed
stage('Test NodeJS Tool')	Required tool for the Allure reporting
stage('Checkout SUT Dev')	pull the dev of SUT
stage('Setup Python Environment (SUT)')	Install equirements.txt pip package for SUT
stage('Setup NodeJS and Newman')	Install Newman for AOI testing and related Nodejs for reporting
stage('Run Unit Tests (SUT)')	Execute unit test cases which located in Repo
stage('Run Integration Tests (SUT)')	Execute integration test cases which located in SUT repo
stage('Build and Deploy SUT to Staging (via Render)')	Deploy the dev to the dev web server from Render free hosting using curl post request
stage('Run API Tests')	Run the API test scripts on the dev service
stage('Run QA Tests (Selenium)')	Run Selenium QA on Dev service
stage('Checkout SUT Main for Merge')	Checkout to main branch of SUT
stage('Merge Dev to Main & Push')	Merge dev branch to main branch
—	Render service will trigger automatically

	to deploy the new commit pushed to main
Post stage	Consolidate each test report to one report and send the result to interested parties

Currently, for our local setup, Jenkins uses polling to detect code changes in the repository. In a real-world production environment, this will transition to webhooks sent directly from the code repository (e.g., GitHub) upon every developer push, ensuring immediate pipeline triggers.

For each testing stage within the pipeline, we generate both a detailed Allure report and a standard JUnit report. A critical aspect of our pipeline's design is its fail-fast mechanism: the failure of any testing stage will immediately terminate all subsequent stages, ensuring rapid feedback and preventing further resource consumption on flawed builds.

We can use many jobs per pipeline instead of single job, it is design decision and need consultant with DevOps

Appendix C : Connecting Concepts

To unlock the full power of test automation at a company, we need to see how critical software delivery concepts like flow, quick feedback, Continuous Integration, Continuous Testing, Continuous Delivery, value stream, value delivery, lead time, and time to market come to life through our team's roles. This section will precisely outline who does what in our new automated delivery pipeline, making individual contributions clear and highlighting how our improved process will boost teamwork and efficiency across every stage of software development.

Flow

smooth, continuous, and efficient movement of work (like features, bug fixes, or new ideas) from its initial concept all the way t, until it reaches the end-user.

- Developers: Initiate and sustain the flow by continuously committing code
- CI Server: Acts as the central orchestrator of the flow. It automatically pulls code, builds the application, runs tests, creates deployable artifacts
- QA Automation Engineers: designing and maintaining automated tests at every stage
- Project Managers: Benefit directly from a predictable and accelerated flow, enabling more accurate planning, reliable release schedules, and overall project control.

Quick Feedback

At the heart of efficient software development is quick feedback. It means rapidly communicating software quality and functionality, so teams can act immediately and fix defects much more cheaply and easily.

- Developers: Receive the most immediate and critical feedback, minutes after push. This eliminates "late feedback cycles."
- QA Automation Engineers: They monitor test results, and ensure that relevant alerts reach the correct teams promptly. They transition from finding bugs late to proactively setting up systems that *tell* developers about bugs early.
- CI Server: Serves as the primary mechanism for delivering quick feedback. It executes tests and immediately publishes results and notifications

- Project Managers: Gain real-time visibility into the health of the codebase and the progress of features through automated test reports, enabling them to make informed decisions swiftly.
- Central Group: Defines and enforces best practices for reporting and feedback mechanisms, ensuring consistency and clarity across different projects.

Continuous Integration (CI)

CI means developers frequently add their code changes to a central hub, and then automated builds and tests run right away to find mistakes fast.

- Developers: Constantly add small code changes to the shared system (like Git). They make sure their changes fit well and fix any issues the CI system flags.
- CI Server: The main engine. It automatically builds the app and runs basic tests every time new code is added. It gives instant feedback on the code's health.
- QA Automation Engineers: Create and keep up the automated tests that run as part of CI. They ensure these tests are strong and give quick, reliable feedback, helping developers merge code confidently.
- Project Managers: Get early warnings about potential problems or code quality issues, leading to more stable and predictable development.

Continuous Testing (CT)

CT is about running automated tests throughout the entire software development process, getting feedback on business risks for a release as early and often as possible.

- QA Automation Engineers: Lead CT. They design, build, and maintain all the automated tests (unit, integration, API, system, performance) that run non-stop through the pipeline. They also set up the system to run these tests automatically and analyze the results.
- CI Server: Makes CT possible by automatically running different automated tests at various points in the pipeline. It collects and reports test results.
- Developers: Get constant feedback from CT, helping them quickly find and fix bugs. This ensures new changes don't break existing features.
- Project Managers: Get real-time updates on app quality and stability, helping them make smart decisions about releasing software and managing risks.

Continuous Delivery (CD)

CD is a way of working where teams release software often and reliably. It builds on CI by making sure all code changes are automatically built, tested, and ready to go live, usually with a final manual check.

- **CI Server:** The core automation power behind CD. It manages everything from new code to automated builds, all levels of testing, creating ready-to-use software packages, and deploying them to different environments (dev, QA, staging). This makes the app ready to deploy anytime.
- **QA Automation Engineers:** Provide the crucial confidence for CD. Their strong and complete automated tests (covering unit, integration, API for Django, and system tests) ensure the software is always high quality, giving certainty that any ready-to-deploy package is truly ready for customers.
- **Developers:** Must ensure their code is always "ready to deploy." This means following coding rules, keeping tests updated, and reacting to automated feedback.
- **Project Managers / Release Managers:** Oversee the CD process, deciding when to push software to customers, relying on the automation and high confidence from continuous testing. They manage the final release timing.
- **Manual QA / UAT Team:** Even with lots of automation, this team might do final, specific manual checks or user acceptance testing (UAT) on a release candidate before it goes live. This ensures the software truly meets business needs.

Value Stream

The value stream covers every activity, useful or not, from a product idea to getting it to the customer. Making the value stream better means making this whole process more effective and efficient.

- **All Roles:** Everyone (Developers, QA Automation Engineers, Project Managers, Manual QA/UAT, CI Server) helps create and shape the value stream. Every step in our CI/CD pipeline, from code changes to live deployment, is part of this stream.
- **Project Managers:** Are key to seeing, managing, and constantly improving the entire value stream. They find problems and ways to get better, tracking progress to keep things flowing smoothly from idea to finished product.
- **Developers:** Create and improve the main product within the value stream, ensuring code quality from the start.

- QA Automation Engineers: Play a big role in speeding up the "quality" part of the value stream. By automating tests, they ensure the product moves through quality checks efficiently and reliably, reducing rework and delays. They actively remove slow manual testing waits.
- CI Server: Automates moving between different value stream stages (build, test, deploy). It's the engine that keeps things moving and cuts down on waiting time.

Value Delivery

Value delivery is about actually giving customers real benefits, features, and solutions. It's the end result of an efficient value stream, turning our internal work into happy customers and business growth.

- Developers: Directly deliver value by building new features, improving existing ones, and fixing bugs that make the user experience better.
- QA Automation Engineers: Are vital for delivering high-quality value. By making sure software is reliable and stable, they guarantee that new features work as promised and meet customer needs. This directly adds to what customers see as valuable and makes them happier. Their work ensures that every release consistently pushes out valuable, stable software.
- Project Managers: Help and track whether value is being delivered according to business goals and customer needs. They prioritize features that will bring the most value and manage the process for timely releases.
- Customers: Are the ones who finally get the value, enjoying the benefits of new features, better performance, and fewer bugs.

Lead Time

Lead time is a key measure that tracks the total time from when an idea or request starts (or a customer orders something) until it's successfully delivered to the customer. Making lead time shorter is a main goal of agile and DevOps methods.

- All Roles: Everyone is responsible for cutting down lead time by making their own work better.
- Developers: Help by writing clean, efficient code and reacting fast to feedback, reducing the time spent on coding and fixing bugs.
- QA Automation Engineers: Dramatically impact lead time by making testing cycles much shorter. By finding bugs earlier (through "Shift-Left" testing and Continuous Testing) and automating repeated

tests, they stop expensive rework and delays later in the process. This directly shrinks the time from development to release.

- CI Server: Is crucial for cutting lead time by automating build, test, and deployment processes (Continuous Integration and Continuous Delivery). It removes manual waiting times and speeds up the whole pipeline.
- Project Managers: Watch and analyze lead time data to find bottlenecks, see how well improvements are working, and drive efforts to make delivery faster.

Time to Market (TTM)

Time to Market (TTM) is a strategic measure that tracks the time from when a product idea starts (or major feature development begins) until it's released and available to end-users or the target market. It's closely related to lead time but is often viewed at a higher, strategic level.